Graphic Era Hill University
DEHRADUN • BHIMTAL • HALDWANI

PROJECT AND TEAM INFORMATION

Project Title

(Try to choose a catchy title. Max 20 words).

Campus Complaint Management Portal: A DBMS and OS-Based Complaint Tracking System

Student/Team Information

Team Name: aakhil1490497

Team member 1 (Team Lead)
Name - Akhil Rana
University Roll No. - 2419417
Section - ML1
Email - aakhil1490497@gmail.com

Team member 2
Name - Abhishek Singh Bisht
University Roll No. - 2419301
Section - DS1
Email - abhishekbisht425@gmail.com

Team member 3
Name - Akshat Gairola
University Roll No. - 2418215
Section - CS1
Email - akshatgairola686@gmail.com

Team member 4
Name - Krishna Barua
University Roll No. - 2418603
Section - E1
Email - baruakrishna426@gmail.com

Team member 5
Name - Ansh Rajput
University Roll No. - 2418288
Section - B1
Email - anshrajput2442@gmail.com

Team member 6
Name - Nalin Bhadola
University Roll No. - 2419208
Section - DS1
Email - nalinbhadola17@gmail.com

PROJECT PROGRESS DESCRIPTION

Project Abstract

(Brief restatement of your project's main goal. Max 300 words).

The Campus Complaint Management Portal is a Django-based web application backed by PostgreSQL that digitizes campus maintenance complaints. Students, faculty, and staff can submit complaints under categories such as Electrical, Wi-Fi, Plumbing, Cleaning, Classroom, and Hostel, while authorized users can assign work, update status, and track progress. An administrative dashboard summarizes complaint counts and related statistics. The project is designed to demonstrate both Database Management System (DBMS) and Operating System (OS) concepts in a practical campus workflow. Its modules cover role-based authentication, complaint submission, transactional assignment with row locking, secure attachment handling, dashboard analytics and scheduling demonstrations, and status tracking with file-based caching. Supporting scripts are included for database backup and process health checks. The codebase and setup notes are present in the supplied project archive. Team details, a public repository URL, and comprehensive automated test results were not provided, so these are left for confirmation rather than assumed.

Updated Project Approach and Architecture

(Describe your current approach, including system design, communication protocols, libraries used, etc. Max 300 words).

The application follows Django's Model–View–Template architecture. Browser requests are routed through campus_portal/urls.py to six feature apps: automation_auth, submission_monitoring, concurrency_transactions, file_engine, analytics_scheduling, and status_caching. Django views implement authentication, complaint submission/listing, assignment, attachment upload/download, dashboard reporting, scheduling demonstrations, and status tracking. HTML templates provide the user interface. PostgreSQL stores users, complaints, assignments/workload data, attachments metadata, and cache metrics; Django ORM is used for data access. The custom CampusUser model is configured as AUTH_USER_MODEL. Assignment uses transaction.atomic() and select_for_update() to lock complaint/workload records during updates. The file engine uses authenticated views and Django file responses for attachment access, while SQL trigger definitions are supplied in file_engine/sql/triggers.sql. Status tracking uses Django's file-based cache with a five-minute timeout and invalidation on relevant updates. Shell scripts are provided for nightly database backup and periodic process health checks, intended to be scheduled with cron. Dependencies listed are Django 5.2.12 and psycopg2-binary 2.9.12. The documented database is PostgreSQL and the project is run through Django's development server. No separate external communication protocol beyond standard browser-to-Django HTTP requests is specified in the project notes.

Tasks Completed

(Describe the main tasks that have been assigned and already completed. Max 250 words).

Task Completed	Team Member
Project topic selection and requirement analysis	Akhil Rana
Designed the system architecture and database structure	Abhishek Singh Bisht
Developed the user registration and login module	Akshat Gairola
Implemented the complaint submission module	Krishna Barua
Developed complaint tracking and status management	Ansh Rajput
Created the admin module for managing complaints	Nalin Badola
Integrated the frontend with backend functionality	Abhishek Singh Bisht

Challenges/Roadblocks

(Describe the challenges that you have faced or are facing so far and how you plan to solve them. Max 300 words).

The project notes record several setup blockers that were addressed: the virtual environment was missing; settings.py lacked required Django configuration and a SECRET_KEY; the custom CampusUser model was not configured as AUTH_USER_MODEL; the database had not been migrated; complaint categories were not seeded; and no admin account existed. The notes state that a virtual environment and dependencies were installed, settings were completed, migrations were applied, six initial categories were loaded, and a superuser was created. The notes also report that Django's system check returned no issues and the login page returned HTTP 200. Remaining technical risks include validating PostgreSQL trigger behavior on the target database, verifying concurrent assignment/status updates, confirming attachment access controls, and testing cache invalidation and scheduled scripts. The automated test files currently contain only Django TestCase placeholders, so repeatable test cases still need to be written and executed. The setup notes include development credentials and a development SECRET_KEY; these should be replaced with secure environment-based values before any deployment beyond local development.

Tasks Pending

(Describe the main tasks that you still need to complete. Max 250 words).

Task Pending	Team Member (to complete the task)
Write and run automated tests for authentication, permissions, complaint CRUD, assignment rollback/locking, attachments, dashboard, and cache invalidation	Abhishek Bisht
Validate SQL triggers and database behavior on a clean PostgreSQL setup	Nalin Badola
Test backup and health-check scripts, then verify cron configuration and logs	Krishna Barua
Run end-to-end scenarios for student submission → admin assignment → staff status update → student tracking	Akshat Gairola
Review security settings, remove development credentials/ keys, and document deployment configuration	Krishna Barua
Add actual team details, repository URL, branch name, and commit references to this report	Akhil Rana

Project Outcome/Deliverables

(Describe what are the key outcomes / deliverables of the project. Max 200 words).

The expected deliverable is a working campus complaint portal with authentication and role-based access, complaint submission and listing, assignment and status updates, attachment handling, a dashboard, and complaint tracking. The codebase also demonstrates DBMS concepts through PostgreSQL persistence, ORM queries, transactional updates, row locking, and a supplied trigger script. OS-related components include file I/O and access controls, priority/SJF scheduling demonstrations, file-based caching, and shell scripts for database backup and process monitoring. Additional deliverables are Django templates, app routing, migrations, category fixtures, setup instructions, and this Phase 2 progress report. Automated test coverage, verified cron execution, and deployment-ready security configuration remain to be completed or confirmed.

Progress Overview

(Summarize how much of the project is done, what's behind schedule, what's ahead of schedule. Max 200 words.)

The supplied archive contains the main Django project structure and all six feature modules, their views/models/URLs, templates, migrations, setup documentation, SQL trigger file, and two OS-oriented shell scripts. The accompanying problem report documents that key environment and configuration blockers were fixed, migrations and category seeding were performed, and the login endpoint and Django system check were verified at that time. However, the automated tests are still placeholders and no test execution report or public repository/commit history was supplied. Therefore, implementation is substantially scaffolded, but functional validation and security hardening should be treated as in progress. No schedule baseline was provided, so ahead-of-schedule or delayed status cannot be reliably assessed.

Codebase Information

(Repository link, branch, and information about important commits.)

Project: Campus_Complain_Management_Portal-main (supplied ZIP archive). Framework: Django 5.2.12;

database adapter: psycopg2-binary 2.9.12;

database: PostgreSQL.

Main entry point: manage.py.

Project configuration and routing: campus_portal/settings.py and campus_portal/urls.py.

Feature apps: automation_auth, submission_monitoring, concurrency_transactions, file_engine, analytics_scheduling, status_caching. SQL trigger definitions: file_engine/sql/triggers.sql.

Setup instructions: README.md; issue/fix notes: problem.txt.

Repository URL : https://github.com/abhisheksinghbist-altt/Campus_Complain_Management_Portal

Testing and Validation Status

(Provide information about any tests conducted)

Test Type	Status (Pass/Fail)	Notes
Django system check	Pass (reported in problem.txt)	The project notes report: no issues (0 silenced) after settings/auth model fixes.
Database migrations	Reported complete	The notes state 26 migrations were applied; not independently rerun for this report.
Initial category fixture	Reported complete	The notes state six category objects were loaded.
Login endpoint smoke check	Pass (reported in problem.txt)	The notes report /auth/login/ returned HTTP 200.
Automated Django test suite	Not yet validated	All six apps contain placeholder TestCase files; no meaningful test cases were found.
Concurrency, attachments, triggers, cache, cron scripts	Pending validation	Dedicated integration and regression tests still need to be implemented and run.

Deliverables Progress

(Summarize the current status of all key project deliverables mentioned earlier. Indicate whether each deliverable is completed, in progress, or pending.)

Authentication and role-based access: Implemented in code; needs test coverage
Complaint submission and listing: Implemented in code; needs end-to-end validation
Transactional assignment and workload updates: Implemented in code; concurrency/rollback tests pending
Attachment upload/download and SQL triggers: Views/script included; security and trigger tests pending
Dashboard and scheduling demonstration: Views/templates included; output validation pending
Status tracking and file-based cache: Implemented in code; cache hit/invalidation tests pending
Database backup and process health scripts: Scripts included; execution/cron verification pending
Automated tests and deployment hardening: Pending